ПРОГРАММИРОВАНИЕ НА C#

События, генерики, LINQ

event
<out T>
yield

Ключевое слово event, вариантность и PECS, отложенное исполнение запросов, yield и деревья выражений

Юрий Уфимцев

О чём поговорим

01

События и делегаты

Ключевое слово event, add и remove, Delegate.Combine, события в Qt и Java

02

Генерики

Теория типов, вариантность и PECS в C#, Java и Kotlin

03

LINQ

Отложенное исполнение, yield, IQueryable, деревья выражений

04

Практика

Вариантность, запросы LINQ, вывод программ

01

События и делегаты

Ключевое слово event, аксессоры add и remove, события в Qt и Java

СОБЫТИЯ

Вспомним про делегаты

```
public delegate void ArrowHandler();

ArrowHandler handler = game.OnLeft;
handler += logger.LeftPressed;  // теперь в цепочке два метода
handler();  // вызовутся оба, в порядке подписки

Action left = game.OnLeft;
Console.WriteLine(left.Target);  // game - у кого вызывать
Console.WriteLine(left.Method.Name);  // OnLeft - что вызывать
```

Значение

Делегат можно положить в переменную, передать, вернуть

Типобезопасность

Сигнатуру метода проверяет компилятор

Цепочка делегатов (delegate chaining)

+= собирает цепочку, один вызов вызывает всех

СОБЫТИЯ

Недостатки публичного делегата

```
public class EventLoop
{
    public Action? LeftHandler { get; set; }
}

loop.LeftHandler += game.OnLeft;
loop.LeftHandler += logger.LeftPressed;

loop.LeftHandler = null;
// все подписчики потеряны

loop.LeftHandler?.Invoke();
// событие вызвано извне класса
```

= вместо +=

Присваивание вместо += удаляет всех подписчиков без предупреждения

Вызов извне

Событие может вызвать любой внешний код

Нарушение инкапсуляции

Класс не контролирует собственный список подписчиков

Вызов делегата: () и ?.Invoke()

Прямой вызов

```
Action? handler = null;
handler();  // NullReferenceException
```

Безопасный вызов

```
Action? handler = null;
handler?.Invoke();  // ничего не происходит
```

Почему не if (handler != null) handler()

Не атомарно, нужен lock

СОБЫТИЯ

Ключевое слово event

```
public class EventLoop
{
    public event Action? LeftHandler;

    public void Run()
    {
        // внутри класса допустимо
        LeftHandler?.Invoke();
    }
}
```

```
loop.LeftHandler += game.OnLeft;  // допустимо
loop.LeftHandler -= game.OnLeft;  // допустимо
loop.LeftHandler = null;  // CS0070
loop.LeftHandler?.Invoke();  // CS0070
```

Проверяет компилятор

CS0070: вне объявившего класса событие допустимо только слева от += или −=

Наследникам вызов недоступен

Поэтому принято объявлять protected virtual void OnLeftPressed(...)

Во что компилятор превращает event

Исходный код

```
public event Action? LeftHandler;
```

Что генерирует компилятор (упрощённо)

```
private Action? leftHandler;  // скрытое поле

public void add_LeftHandler(Action? value)
    ⇒ leftHandler = (Action?)Delegate.Combine(leftHandler, value);

public void remove_LeftHandler(Action? value)
    ⇒ leftHandler = (Action?)Delegate.Remove(leftHandler, value);
```

Как у свойства: скрытое поле и пара методов. Оператор += вне класса вызывает add_LeftHandler, оператор −= вызывает remove_LeftHandler.

СОБЫТИЯ

+= и −= вызывают Delegate.Combine и Delegate.Remove

```
// DELEGATE
Action? handler = null;

handler += game.OnLeft;
// компилируется в:
handler = (Action?)Delegate.Combine(handler, game.OnLeft);

handler -= game.OnLeft;
// компилируется в:
handler = (Action?)Delegate.Remove(handler, game.OnLeft);

// EVENT
// += и -= - вызовы add и remove:
loop.LeftHandler += game.OnLeft;  // loop.add_LeftHandler(game.OnLeft)
// внутри add - тот же Delegate.Combine
```

EVENT: **+=** → **add** → **Delegate.Combine**. Для простого delegate шаг с add пропускается.

СОБЫТИЯ

Как ведёт себя цепочка делегатов

Ситуация	Что происходит
Combine и Remove	Делегаты неизменяемы: каждая операция создаёт новый объект
Повторная подписка	Метод будет вызван дважды
Remove	Удаляет последнее вхождение; если метода нет, цепочка не меняется
Удаление последнего	Результат - null, поэтому вызов выполняется через ?.Invoke()
–= game.OnLeft	Делегаты сравниваются по паре Target и Method, поэтому новый делегат находит ранее добавленный
Цепочка с результатом	Возвращается результат последнего делегата
Исключение в обработчике	Остальные обработчики не вызываются; решение - GetInvocationList() и try/catch

add и remove вручную

```
// System.ComponentModel
private readonly EventHandlerList events = new();
private static readonly object LeftKey = new();

public event EventHandler LeftPressed
{
    add ⇒ events.AddHandler(LeftKey, value);
    remove ⇒ events.RemoveHandler(LeftKey, value);
}

protected virtual void OnLeftPressed(EventArgs e)
    ⇒ (events[LeftKey] as EventHandler)?.Invoke(this, e);
```

Применение

- Разреженное хранение: у элемента управления WinForms десятки событий, но подписки есть у немногих - один словарь вместо десятков полей
- Перенаправление подписки во вложенный объект
- Проверки и журналирование при подписке
- Слабые подписки (weak events)

Скрытое поле не создаётся: вызвать LeftPressed(...) напрямую нельзя

Переопределение в наследнике: событие и делегат

Событие: add и remove

```
public override event Action? MyEvent
{
    add
    {
        Console.WriteLine("Лог подписки");
        base.MyEvent += value;
    }
    remove ⇒ base.MyEvent -= value;
}

obj.MyEvent += h;
// вызывается только add
```

Свойство-делегат: get и set

```
public override Action? MyDelegate
{
    get ⇒ base.MyDelegate;
    set
    {
        Console.WriteLine("Лог изменения");
        base.MyDelegate = value;
    }
}

obj.MyDelegate += h;
// компилятор разворачивает в
obj.MyDelegate = obj.MyDelegate + h;
// вызываются и get, и set
```

СОБЫТИЯ

Преимущества add и remove перед get и set

Критерий	event с add и remove	Свойство-делегат
Инкапсуляция вызова	Вызвать событие может только объявивший его класс; наследник этого правила нарушить не может	Любой код, получивший делегат через get, может вызвать его в обход логики базового класса
Присваивание =	Вне класса запрещено (CS0070)	Разрешено и удаляет всех подписчиков
Перехват	add и remove получают конкретный обработчик	set получает готовую цепочку: неизвестно, была это подписка или отписка
Потокобезопасность	Сгенерированные add и remove атомарны; при перенаправлении в base это сохраняется	+= состоит из get, Combine и set; lock только внутри set не помогает, блокировка должна охватывать всю операцию

ДРУГИЕ ПЛАТФОРМЫ · C++

Qt: сигналы и слоты

```
class EventLoop : public QObject {
    Q_OBJECT
signals:
    void leftPressed();
};

class Game : public QObject {
    Q_OBJECT
public slots:
    void onLeft();
};

connect(&loop, &EventLoop::leftPressed,
        &game, &Game::onLeft);
emit loop.leftPressed();
```

Преимущества

- Слот выполняется в потоке получателя
- Автоматический разрыв при удалении

Недостатки по сравнению с C#

- Требуется генератор кода moc
- Только наследники QObject
- **Любой код может вызвать сигнал (в отличие от C# event)**

ДРУГИЕ ПЛАТФОРМЫ · JAVA

Spring: @EventListener

```
record LeftPressed() {}

@Component
class EventLoop {
    private final ApplicationEventPublisher pub;
    EventLoop(ApplicationEventPublisher pub) {
        this.pub = pub;
    }
    void onKey() { pub.publishEvent(new LeftPressed()); }
}

@Component
class Game {
    @EventListener
    void onLeft(LeftPressed event) { /* ... */ }
}
```

Принцип работы

- Подписка по типу события
- Поиск подписчиков рефлексией
- Издатель не знает подписчиков

Недостатки по сравнению с C#

- Работает только для бинов Spring
- Неявная связь издателя и подписчиков
- Динамическая отписка не предусмотрена

ДРУГИЕ ПЛАТФОРМЫ · JAVA

Java без фреймворков: список подписчиков вручную

```
interface ArrowListener { void onArrow(); }

class EventLoop {
    private final List<ArrowListener> left = new ArrayList<>();
    void addLeft(ArrowListener l) { left.add(l); }
    void removeLeft(ArrowListener l) { left.remove(l); }
    void fireLeft() { for (var l : left) l.onArrow(); }
}

loop.addLeft(game::onLeft);
loop.addLeft(() -> log.add("left"));
loop.removeLeft(game::onLeft);  // подписка не удалится
```

Нет delegate chaining

Для каждого события нужны свой список и три метода: add, remove, fire

Проблема отписки

Каждое выражение game::onLeft создаёт новый объект без equals, поэтому ссылку на него нужно сохранить

ДРУГИЕ ПЛАТФОРМЫ

Сравнение подходов

	C# event	Qt	Spring	Java вручную
Реализация	Язык	Язык и генератор moc	Фреймворк	Код приложения
Кто может вызвать	Только объявивший класс	Любой код	Любой код с доступом к publisher	Зависит от реализации
Подписка	+=	connect(...)	@EventListener	addXxx(...)
Отписка	−= с тем же методом	disconnect или автоматически при удалении	Не предусмотрена	Только по сохранённой ссылке
Несколько подписчиков	Встроено	Встроено	Встроено	Список вручную

02

Генерики

Теория типов, частичный порядок, PECS и вариантность в C#, Java и Kotlin

Тип задаёт интерпретацию данных

В памяти хранится байт 0x41. Это число 65, символ A или часть машинной инструкции?

Тип определяет **интерпретацию** значения и **набор допустимых операций**. Система типов запрещает бессмысленные операции, например сложение числа с функцией.

Без типов противоречивой оказывается даже математика: парадокс Рассела в наивной теории множеств.

Статическая

Типы известны при компиляции: C#, Java, Kotlin

Динамическая

Типы проверяются при исполнении: Python, JavaScript

Сильная

Неявных небезопасных преобразований нет: в Python "1" + 1 - ошибка

Слабая

Типы приводятся неявно: в JavaScript "1" + 1 - это "11"

Полиморфизм: одна операция, много типов

Ad-hoc

Конечный набор типов, для каждого своя реализация

Перегрузка

```
Math.Abs(-1); Math.Abs(-1.5);
```

Разные методы под одним именем

Приведение

```
1 + 1.0
```

int неявно становится double

Универсальный

Бесконечно много типов, один и тот же код

Подтипы

```
void Feed(Animal a); Feed(cat);
```

Наследник подходит везде, где ждут предка

Параметрический

```
T Max<T>(T a, T b)
```

Тип - параметр, его подставят при вызове. Это и есть генерики

Элементарные типы, конструкторы и равенство типов

Элементарные типы

`int · bool · char · double`

Конструкторы типов

Строят новые типы из имеющихся

`T[] · T? · (T1, T2) · Func<T, R> · class`

Выражение над типами

`Dictionary<string, List<(int, bool)>>`

По имени (номинальное): типы равны, если совпадают имена. **Структурное**: если совпадает устройство.

```
record A(int X);
record B(int X);
A a = new B(1);  // ошибка: по имени (record)

(int X, string Y) p = (1, "a");
(int A, string B) q = p;  // допустимо: структурно (кортеж)

delegate int F(int x);
Func<int, int> f = x ⇒ x;
F g = f;  // ошибка: тоже по имени (delegate)
```

Генерик как функция над типами

List : T ↦ List<T>

Генерик	«Сигнатура»	Применение
List<T>	Тип → Тип	List<int>, List<string>
Dictionary<K, V>	Тип × Тип → Тип	Dictionary<string, int>
Func<T, R>	Тип × Тип → Тип	Func<int, bool>

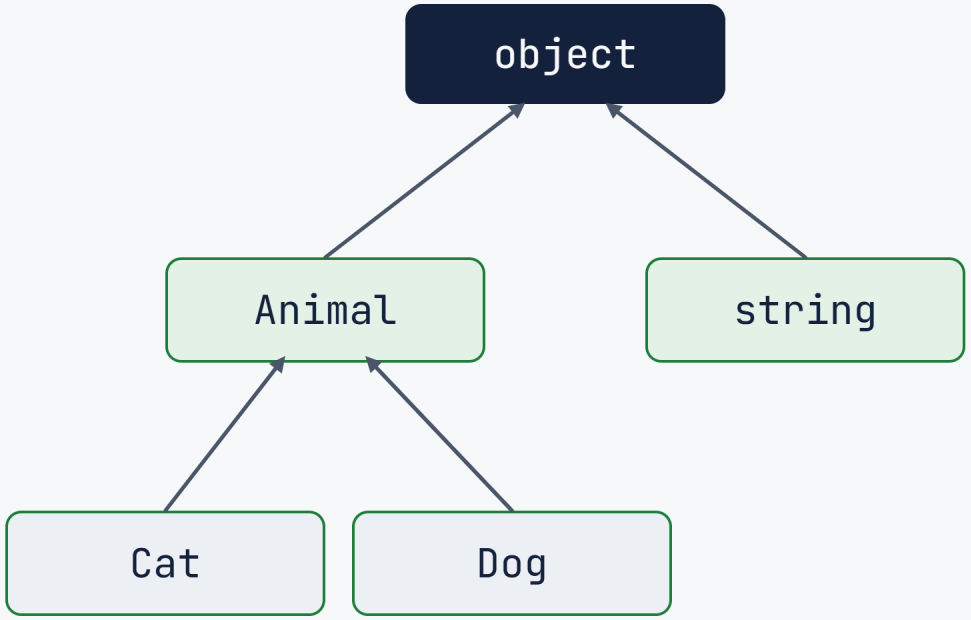
```
typeof(List<>)    // открытый тип: функция, ещё не применённая
typeof(List<int>) // закрытый тип: результат применения
```

Подтипы: частичный порядок на типах

A ≤ B: A - подтип B, B - надтип A. Где ждут B, можно передать A.

- Рефлексивность: A ≤ A
- Транзитивность: Cat ≤ Animal ≤ object ⇒ Cat ≤ object
- Антисимметричность: A ≤ B и B ≤ A ⇒ A = B

Частичный: Cat и Dog несравнимы. object - наибольший элемент.



Надтипы и подтипы

- У типов в Java определён частичный порядок **A > B**: A является надтипом B, B является подтипом A. Запись **A :> B** допускает равенство A = B.
- **A :> B** рефлексивно, антисимметрично и транзитивно: это рефлексивно-транзитивное замыкание отношения $\mathbf{A} >_1 \mathbf{B}$ (прямой надтип).
- **A > B** антирефлексивно, антисимметрично и транзитивно.
- Надтип не должен предоставлять больше возможностей, чем предоставляет подтип.

$$S :> T \not\Rightarrow X<S> :> X<T>$$

Из отношения между типами в общем случае не следует такое же отношение между генериками.

Сохраняет ли генерик порядок на типах?

`Cat ≤ Animal ⇒ G<Cat> ? G<Animal>`

Ковариантность

`G<Cat> ≤ G<Animal>`

Порядок сохраняется

IEnumerable<T>, Func<TResult>

Контравариантность

`G<Animal> ≤ G<Cat>`

Порядок переворачивается

IComparer<T>, Action<T>

Инвариантность

`G<Cat> и G<Animal>`

Никак не связаны

List<T>

Почему List<Cat> не является List<Animal>

```
List<Cat> cats = new() { new Cat() };
List<Animal> animals = cats;  // допустим, компилятор разрешил

animals.Add(new Dog());  // собака - тоже животное
Cat cat = cats[1];  // в списке кошек оказалась собака
```

Чтение безопасно

Извлечённая кошка является животным: для чтения нужна ковариантность

Запись небезопасна

В список кошек попадает собака: для записи нужна контравариантность

Чтение и запись

Допустима только инвариантность

ГЕНЕРИКИ · ВАРИАНТНОСТЬ

PECS: Producer Extends, Consumer Super

Producer

Объект **возвращает** значения типа T, из него только читают

Java	? extends T
C#, Kotlin	out T
Операция	Чтение
Вариантность	Ковариантность
Пример	IEnumerable<out T>

Consumer

Объект **принимает** значения типа T, в него только записывают

Java	? super T
C#, Kotlin	in T
Операция	Запись
Вариантность	Контравариантность
Пример	IComparer<in T>

Правило Джошуа Блоха из книги «Effective Java». В C# и Kotlin оно выражено ключевыми словами out и in.

Producer: ковариантность, out

```
// из стандартной библиотеки: T только читаем
public interface IEnumerable<out T> { IEnumerator<T> GetEnumerator(); }
void PrintAll(IEnumerable<Animal> animals)
{
    foreach (var a in animals)
        Console.WriteLine(a.Name);
}
List<Cat> cats = GetCats();
PrintAll(cats);  // компилируется
// шаг 1: List<Cat> → IEnumerable<Cat> - List реализует интерфейс, вариантность не нужна
// шаг 2: IEnumerable<Cat> → IEnumerable<Animal> - ковариантность, out T
```

Поведение задано в методе, мы под него подстраиваемся

Consumer: контравариантность, in

```
// из стандартной библиотеки: T только пишем
public interface IComparer<in T> { int Compare(T? x, T? y); }

class ByWeight : IComparer<Animal>
{
    public int Compare(Animal? x, Animal? y) ⇒ x!.Weight.CompareTo(y!.Weight);
}

List<Cat> cats = GetCats();
cats.Sort(new ByWeight());  // ожидается IComparer<Cat>, передан IComparer<Animal>
```

Поведение задаём мы сами, и важно задать его безопасно



ГЕНЕРИКИ · ВАРИАНТНОСТЬ

PECS в картинке

Контравариантность, слева. Нужно избавиться от бамбука: подойдёт любой потребитель, принимающий бамбук, в том числе тот, кто сжигает всё.

Ковариантность, справа. Нужна музыка: подойдёт производитель музыки или более конкретный производитель метала.

Источник:
https://stackoverflow.com/questions/2723397/what-is-pecs-producer-extends-consumer-super/19739576#19739576

C#: in и out в объявлении типа

Стандартная библиотека

```
interface IEnumerable<out T>
interface IComparer<in T>
delegate void Action<in T>(T obj)
delegate TResult Func<in T, out TResult>(T arg)

Func<object, string> f = o ⇒ o.ToString()!;
Func<string, object> g = f;  // допустимо
```

Компилятор проверяет позиции T

```
interface IBox<out T>
{
    T Get();  // допустимо: T на выходе
    void Put(T item);  // CS1961: T на входе
}
```

Только интерфейсы и делегаты

Классы и структуры всегда инвариантны: List<T>, Tuple<T1, T2>

Только ссылочные типы

IEnumerable<int> не приводится к IEnumerable<object>: потребовалась бы упаковка каждого элемента

Java: вариантность в месте использования

```
// List<T> инвариантен; вариантность выбирают в сигнатуре метода
static double totalWeight(List<? extends Animal> animals) {
    double sum = 0;
    for (Animal a : animals) sum += a.weight();  // читать можно
    // animals.add(new Cat());  писать нельзя: ошибка компиляции
    return sum;
}

static void addCats(List<? super Cat> sink) {
    sink.add(new Cat());  // писать можно
    Object o = sink.get(0);  // читать - только как Object
}

// java.util.Collections
public static <T> void copy(List<? super T> dest, List<? extends T> src)
```

Kotlin: и declaration-site, и use-site

В объявлении, как в C#

```
interface Source<out T> {
    fun next(): T
}

interface Comparable<in T> {
    operator fun compareTo(other: T): Int
}

// List только для чтения: List<out E>
val cats: List<Cat> = listOf(Cat())
val animals: List<Animal> = cats  // допустимо
```

В использовании, как в Java

```
// Array<T>, MutableList<T> инвариантны
fun copy(from: Array<out Any>,
         to: Array<Any>) {
    for (i in from.indices)
        to[i] = from[i]
}

fun fill(dest: Array<in String>,
         value: String) {
    for (i in dest.indices)
        dest[i] = value
}
```

Use-site variance в Kotlin называется **type projections**.

ГЕНЕРИКИ · ВАРИАНТНОСТЬ

Ковариантность массивов

Массивы допускают запись, но в C# и Java они ковариантны. Ошибка типов, которую должен был обнаружить компилятор, проявляется только во время выполнения.

```
string[] strings = new string[1];
object[] objects = strings;  // компилируется: массивы ковариантны
objects[0] = 42;  // ArrayTypeMismatchException
```

Наследие Java

Поведение заимствовано из Java, где массивы ковариантны с первой версии

Проверка при записи

Каждая запись в массив ссылочного типа проверяет тип элемента

Kotlin

Array<T> инвариантен: ошибка обнаруживается при компиляции

Сравнение C#, Java и Kotlin

	C#	Java	Kotlin
Где задаётся вариантность	В объявлении: in, out	В использовании: ? extends, ? super	В объявлении и в использовании
Вариантные типы	Интерфейсы и делегаты	Любые генерики через wildcard	Классы и интерфейсы
Изменяемые коллекции	Инвариантны	Инвариантны	Инвариантны
Коллекции только для чтения	IEnumerable<out T>, IReadOnlyList<out T>	Отдельных интерфейсов нет	List<out E>
Массивы	Ковариантны, проверка при выполнении	Ковариантны, проверка при выполнении	Инвариантны

03 LINQ

Language-Integrated Query: методы, отложенное исполнение, язык запросов, yield, IQueryable и деревья выражений

LINQ

Что такое LINQ и зачем он нужен

Императивный подход

```
var result = new List<string>();
foreach (var s in students)
{
    if (s.Group == "25.Б41")
        result.Add(s.Name.ToUpper());
}
result.Sort();
```

С использованием LINQ

```
var result = students
    .Where(s => s.Group == "25.Б41")
    .Select(s => s.Name.ToUpper())
    .OrderBy(name => name);
```

Декларативность

Описывается требуемый результат, а не способ обхода коллекции

Единый синтаксис

Одни и те же операции для коллекций, XML и баз данных

Интеграция в язык

Запрос проверяется компилятором, в отличие от SQL в строке

LINQ как набор методов-расширений

```
// System.Linq.Enumerable
public static IEnumerable<TSource> Where<TSource>(
    this IEnumerable<TSource> source,
    Func<TSource, bool> predicate)

students.Where(s ⇒ s.Age > 18)
// компилятор преобразует в вызов статического метода:
Enumerable.Where(students, s ⇒ s.Age > 18)
```

Генерики

`Where<TSource>`

Один метод для коллекций любого типа

Делегаты и лямбды

`Func<TSource, bool>`

Условие передаётся как значение

Методы-расширения

`this IEnumerable<TSource>`

Вызов через точку без изменения типа

LINQ

Основные методы

Категория	Методы
Фильтрация	Where, OfType
Проекция	Select, SelectMany, Zip
Сортировка	OrderBy, OrderByDescending, ThenBy, Reverse
Разбиение	Take, Skip, TakeWhile, SkipWhile, Chunk
Множества	Distinct, Union, Intersect, Except и их варианты ...By
Группировка и соединение	GroupBy, ToLookup, Join, GroupJoin
Квантификаторы	Any, All, Contains
Агрегирование	Count, Sum, Min, Max, MinBy, MaxBy, Average, Aggregate
Получение элемента	First, Last, Single, ElementAt и их варианты ...OrDefault
Материализация	ToList, ToArray, ToDictionary, ToHashSet

Данные для примеров

```
record Student(string Name, string Group, int Age);
record Grade(string StudentName, string Course, int Score);

List<Student> students =
[
    new("Анна", "25.Б41", 19), new("Борис", "25.Б41", 20),
    new("Вера", "25.Б42", 18), new("Глеб", "25.Б42", 19),
    new("Дина", "25.Б43", 20), new("Егор", "25.Б43", 18),
];

List<Grade> grades =
[
    new("Анна", "C#", 5), new("Анна", "Матан", 4), new("Борис", "C#", 3),
    new("Вера", "C#", 5), new("Вера", "Матан", 5), new("Глеб", "C#", 4),
    new("Дина", "Матан", 4),
];
```

LINQ · ПРИМЕРЫ

Фильтрация, проекция, агрегирование

```
// Студенты группы 25.Б42 в алфавитном порядке
var b42 = students.Where(s => s.Group == "25.Б42").OrderBy(s => s.Name);

// Оценки по C# в виде строк «Анна: 5»
var csharp = grades.Where(g => g.Course == "C#").Select(g => $"{g.StudentName}: {g.Score}");

// Квантификаторы
bool anyTwo = grades.Any(g => g.Score == 2);  // false
bool allAdults = students.All(s => s.Age >= 18);  // true

// Агрегирование
double average = grades.Average(g => g.Score);  // 4.29
var best = grades.Where(g => g.Course == "C#").MaxBy(g => g.Score);  // Анна, 5
```

Группировка и соединение

```
// Средний балл по каждому курсу
var byCourse = grades
    .GroupBy(g ⇒ g.Course)
    .Select(gr ⇒ new { Course = gr.Key, Average = gr.Average(g ⇒ g.Score) });
// C#: 4.25, Матан: 4.33

// Оценка вместе с группой студента: соединение двух коллекций
var withGroup = grades.Join(students,
    g ⇒ g.StudentName, s ⇒ s.Name,
    (g, s) ⇒ new { s.Name, s.Group, g.Course, g.Score });

// Студенты без оценок
var noGrades = students.ExceptBy(grades.Select(g ⇒ g.StudentName), s ⇒ s.Name);
// Егор
```

SelectMany, Zip, Aggregate, материализация

```
// SelectMany: последовательность последовательностей в одну
string[] phrases = ["an apple a day", "the quick brown fox"];
var words = phrases.SelectMany(p ⇒ p.Split(' '));  // an, apple, a, day, the, …

// Zip: попарное объединение двух последовательностей
int[] numbers = [1, 2, 3];
string[] letters = ["a", "b", "c"];
var pairs = numbers.Zip(letters, (n, l) ⇒ $"{n}{l}");  // 1a, 2b, 3c

// Aggregate: свёртка (fold)
int product = numbers.Aggregate(1, (acc, x) ⇒ acc * x);  // 6

// Материализация в словарь и в таблицу соответствия
var byName = students.ToDictionary(s ⇒ s.Name);
var byGroup = students.ToLookup(s ⇒ s.Group);  // byGroup["25.Б41"]: Анна, Борис
```

LINQ · ОТЛОЖЕННОЕ ИСПОЛНЕНИЕ

Отложенное исполнение

Код

```
var numbers = new List<int> { 1, 2, 3 };
var query = numbers.Where(n ⇒
{
    Console.WriteLine($"проверка {n}");
    return n > 1;
});
numbers.Add(4);
Console.WriteLine(query.Count());
Console.WriteLine(query.Count());
```

Вывод

```
проверка 1 … проверка 4
3
проверка 1 … проверка 4
3
```

Запуск только в Count()

Where только запомнил источник и условие

Учтён новый элемент

Запрос читает источник в момент выполнения

Каждый обход заново

Для повторного использования результата нужен ToList()

LINQ · ОТЛОЖЕННОЕ ИСПОЛНЕНИЕ

Немедленные и отложенные операции

Немедленные

Выполняются сразу и возвращают значение или коллекцию

`Count, Sum, First, ToList, ToArray`

Отложенные потоковые

Выдают элементы по одному, по мере запроса

`Where, Select, Take, Skip`

Отложенные непотоковые

При первом запросе читают весь источник

`OrderBy, GroupBy, Reverse`

Замыкание захватывает переменную, а не её значение

```
int min = 3;
var large = numbers.Where(n ⇒ n > min);
min = 10;
var list = large.ToList();  // фильтрация выполняется с условием n > 10
```

LINQ · ЯЗЫК ЗАПРОСОВ

Встроенный язык запросов

Синтаксис запроса

```
var query =
    from s in students
    where s.Age ⩾ 19
    orderby s.Name
    select s.Name.ToUpper();
```

Синтаксис методов

```
var query = students
    .Where(s ⇒ s.Age ⩾ 19)
    .OrderBy(s ⇒ s.Name)
    .Select(s ⇒ s.Name.ToUpper());
```

Синтаксический сахар

Компилятор переводит запрос в вызовы методов; формы можно сочетать

Структура запроса

Начинается с from, заканчивается select или group; внутри - where, orderby, join, let

Утиная типизация

Достаточно методов Where, Select и других с подходящими сигнатурами

group, join, let

```
// Средний балл по курсам
var byCourse =
    from g in grades
    group g.Score by g.Course into scores
    select new { Course = scores.Key, Avg = scores.Average() };

// Соединение с промежуточной переменной
var withGroup =
    from g in grades
    join s in students on g.StudentName equals s.Name
    let name = s.Name.ToUpper()
    select new { name, s.Group, g.Score };
```

Ключевое слово	Метод
where	Where
select	Select
orderby … descending	OrderByDescending
group … by … into	GroupBy
join … on … equals	Join
let	Select в анонимный тип
второй from	SelectMany

LINQ · РЕАЛИЗАЦИЯ

yield return: как реализован Where

```
public static IEnumerable<T> MyWhere<T>(
    this IEnumerable<T> source,
    Func<T, bool> predicate)
{
    foreach (var item in source)
    {
        if (predicate(item))
        {
            yield return item;
        }
    }
}
```

Конечный автомат

Метод становится классом-перечислителем

Вызов не выполняет тело

MyWhere(...) только создаёт объект-перечислитель

Выполнение по частям

MoveNext() выполняет тело до следующего yield return

LINQ · РЕАЛИЗАЦИЯ

yield break и бесконечные последовательности

```
static IEnumerable<int> Naturals()
{
    for (var i = 1; ; i++)
        yield return i;  // бесконечная последовательность
}

static IEnumerable<T> MyTake<T>(this IEnumerable<T> source, int n)
{
    if (n ⩽ 0) yield break;  // завершить последовательность
   ...
}
```

LINQ · ПРОВАЙДЕРЫ

IEnumerable и IQueryable

IEnumerable<T> · LINQ to Objects

`Where(Func<T, bool> predicate)`

Лямбда-выражение компилируется в делегат и выполняется в процессе приложения

IQueryable<T> · LINQ-провайдеры

`Where(Expression<Func<T, bool>> predicate)`

Лямбда-выражение становится деревом выражений, провайдер транслирует его, например, в SQL

```
IQueryable<Student> q = db.Students.Where(s ⇒ s.Age > 18);
// EF Core: SELECT ... FROM Students WHERE Age > 18

IEnumerable<Student> e = db.Students;
var adults = e.Where(s ⇒ s.Age > 18);
// Enumerable.Where: вся таблица загружается в память и фильтруется там
```

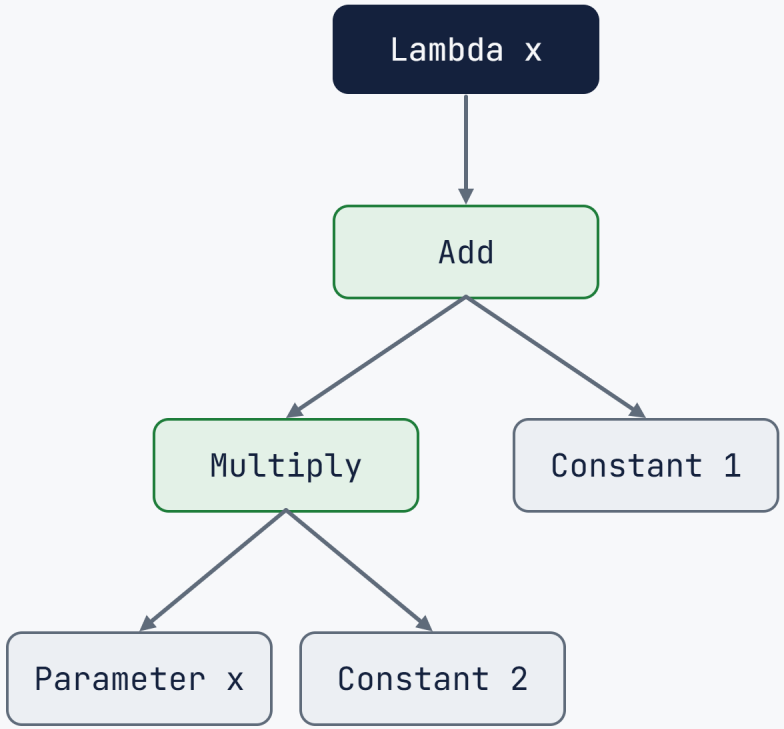
Expression: лямбда-выражение как данные

```
Func<int, int> f = x ⇒ x * 2 + 1;  // код
Expression<Func<int, int>> e = x ⇒ x * 2 + 1;  // дерево

Console.WriteLine(e);  // x ⇒ ((x * 2) + 1)
var body = (BinaryExpression)e.Body;
Console.WriteLine(body.NodeType);  // Add
Console.WriteLine(body.Left);  // (x * 2)
Console.WriteLine(body.Right);  // 1

var compiled = e.Compile();
Console.WriteLine(compiled(5));  // 11
```

Для Expression компилятор строит не код, а дерево объектов. Его можно обойти, проанализировать, перевести на другой язык или скомпилировать.



Разбор дерева: транслятор условия в SQL

```
static string ToSql(Expression e) => e switch
{
    BinaryExpression b => $"({ToSql(b.Left)} {Op(b.NodeType)} {ToSql(b.Right)})",
    MemberExpression m => m.Member.Name,
    ConstantExpression { Value: string s } => $"'{s}'",
    ConstantExpression c => $"{c.Value}",
    _ => throw new NotSupportedException(e.NodeType.ToString())
};

static string Op(ExpressionType t) => t switch
{
    ExpressionType.GreaterThan => ">", ExpressionType.Equal => "=",
    ExpressionType.AndAlso => "AND", _ => throw new NotSupportedException()
};

Expression<Func<Student, bool>> filter = s => s.Age > 18 && s.Group == "25.Б41";
ToSql(filter.Body);  // ((Age > 18) AND (Group = '25.Б41'))
```

Пользовательский LINQ-провайдер

1. Запрос
Цепочка вызовов методов класса Queryable

2. Дерево
Каждый вызов добавляет узел в Expression

3. Выполнение
foreach или ToList() вызывает Provider.Execute

4. Трансляция
Провайдер строит SQL, HTTP-запрос или запрос к MongoDB

```
public interface IQueryProvider
{
    IQueryable<T> CreateQuery<T>(Expression expression);
    TResult Execute<TResult>(Expression expression);
    // и негенерические версии этих методов
}
```

Примеры провайдеров

- Entity Framework Core - SQL
- MongoDB C# Driver - запросы MongoDB
- Клиенты OData - параметры URL

04

Практика

Вариантность, запросы LINQ и отложенное исполнение

ПРАКТИКА · ЗАДАЧА 1

Что выведет программа?

Сколько раз будет вызвано лямбда-выражение внутри Select?

```
int calls = 0;
var squares = Enumerable.Range(1, 3)
    .Select(x ⇒ { calls++; return x * x; });

Console.WriteLine(squares.Sum());
Console.WriteLine(squares.Max());
Console.WriteLine(calls);
```

ПРАКТИКА · ЗАДАЧА 2

Запросы к данным о студентах

Используйте коллекции students и grades со слайда «Данные для примеров». Каждую задачу решите одним запросом LINQ.

1. Имена студентов старше 18 лет в верхнем регистре, по убыванию возраста
2. Количество оценок у каждого студента, включая студентов без оценок
3. Курс с наибольшим средним баллом
4. Студенты, у которых все оценки равны 5
5. Три студента с наибольшим средним баллом
6. Словарь «группа → количество студентов»
7. Средний балл по каждой группе
8. Курсы, по которым все оценки не ниже 4

Дополнительно: запишите задачи 5 и 7 в синтаксисе запросов.

ПРАКТИКА · ЗАДАЧА 3

Что выведет программа?

В какой строке возникнет исключение и возникнет ли оно вообще?

```
var numbers = new List<int> { 1, 2, 0, 4 };
var query = numbers.Select(n ⇒ 10 / n);
Console.WriteLine("запрос построен");

Console.WriteLine(query.First());
Console.WriteLine(query.Take(2).Sum());
Console.WriteLine(query.Count());
```

ПРАКТИКА · ЗАДАЧА 4

Что выведет программа?

Предскажите вывод, не запуская код, и объясните порядок строк.

```
var source = new List<int> { 1, 2, 3, 4, 5 };
int limit = 2;

var query = source
    .Where(n ⇒ { Console.Write($"w{n} "); return n > limit; })
    .Select(n ⇒ { Console.Write($"s{n} "); return n * 10; });

limit = 3;
source.Add(6);

foreach (var x in query.Take(2))
    Console.Write($"[{x}] ");
```

ПРАКТИКА · ЗАДАЧА 5

Какие строки скомпилируются?

Для каждой строки определите, скомпилируется ли она, и объясните почему.

```
IEnumerable<object> a = new List<string>();  // 1
List<object> b = new List<string>();  // 2
Action<string> c = (Action<object>)(o ⇒ { });  // 3
Action<object> d = (Action<string>)(s ⇒ { });  // 4
```

ПРАКТИКА · ЗАДАЧА 6 · ДОПОЛНИТЕЛЬНО

Что выведет программа?

Сравните с задачей 4: в каком порядке будут напечатаны w и элементы?

```
int[] numbers = [5, 3, 8, 1];

var query = numbers
    .Where(n ⇒ { Console.Write($"w{n} "); return n > 2; })
    .OrderBy(n ⇒ n);

foreach (var x in query)
    Console.Write($"[{x}] ");
```

ПРАКТИКА · ЗАДАЧА 7 · ДОПОЛНИТЕЛЬНО

Что выведет программа?

Оба фрагмента строят фильтр «делится на 2 и на 3». Совпадёт ли результат?

```
IEnumerable<int> a = Enumerable.Range(1, 10);
for (int i = 2; i ⩽ 3; i++)
    a = a.Where(x ⇒ x % i == 0);
Console.WriteLine(string.Join(", ", a));

IEnumerable<int> b = Enumerable.Range(1, 10);
foreach (var d in new[] { 2, 3 })
    b = b.Where(x ⇒ x % d == 0);
Console.WriteLine(string.Join(", ", b));
```

Итоги

01

События

event ограничивает доступ к делегату: извне доступны только += и −=.

Операторы += и −= вызывают add и remove, а те вызывают Delegate.Combine и Delegate.Remove.

02

Генерики

Генерик является функцией над типами.

Чтение: out и ковариантность. Запись: in и контравариантность. Чтение и запись: инвариантность.

03

LINQ

Генерики, делегаты и методы-расширения; запрос выполняется отложенно.

IQueryable передаёт провайдеру дерево выражений вместо скомпилированного кода.